

FIRM-WM: State-factorized factual–interventional recurrent modeling for reward-free visual planning

Yilun Wu^{1,2} Yunjian Zhang² Aobo Li²
Mujiangshan Wang^{3,2,*} Haitao Wu¹ Aqiang Zhang^{4,5}

¹Saint Petersburg State University, Saint Petersburg, Russia

²Shenzhen Kaihong Digital Industry Development Co., Ltd., Shenzhen, China

³Chinese Academy of Sciences (CAS) – Shenzhen Institute of Advanced Technology

⁴Harbin Institute of Technology, Harbin, China

⁵Chongqing Research Institute of HIT, Chongqing, China

wuyilun310@gmail.com yunjzhang@xmu.edu.cn

ali09@qub.ac.uk wuhaitao2023@gmail.com

*Corresponding author: Mujiangshan Wang (mjs.wang@siat.ac.cn)

ABSTRACT

Reward-free latent world models can learn from offline videos and solve new image-goal tasks by optimizing actions through predicted latent futures. This setting places two demands on the planning state: its coordinates must be comparable with a goal image. Moreover, its dynamics must retain velocity, motion trend, contact, and other history-dependent information beyond those goal coordinates. Offline training creates a second mismatch: each recorded trajectory reveals one factual future, whereas a sampling-based planner compares many actions that were not taken from the same state. We introduce FIRM-WM (*Factual–Interventional Recurrent World Model*), a compact pixel world model designed around these two gaps. Its recurrent state separates a typed, goal-comparable configuration from a 128-dimensional dynamic fiber used for prediction but excluded from the terminal goal cost. Broad factual trajectories provide state coverage, while common-reset intervention branches provide observed outcomes for alternative action sequences. Before executing each branch, we reset the environment and restore the same recorded values exposed by the environment’s state-setting interface. Under matched CEM planning and three independent full-pipeline seeds, FIRM-WM reaches $99.0 \pm 1.0\%$ on TwoRoom, $92.7 \pm 2.1\%$ on Reacher, and $88.0 \pm 3.0\%$ on OGBench-Cube, compared with 89.0%, 88.0%, and 70.0% for LeWM. The deployed model uses 2.98–3.42 M parameters and records $2.13\text{--}11.60\times$ lower planning time on these tasks.

1 INTRODUCTION

World models promise an appealing separation between learning and acting: learn task-agnostic dynamics from offline experience, then compose those dynamics with a goal at deployment. Recent latent predictive models make this practical from pixels by replacing image reconstruction with prediction in a compact representation space (Hafner et al., 2019; Zhou et al., 2025; Sobal et al., 2025; Maes et al., 2026). Given a current observation and a goal image, these systems roll candidate action sequences through a learned predictor and optimize a latent terminal cost with the cross-entropy method (CEM) (Rubinstein, 1999). They require no task reward during representation learning and plan directly with the learned world model at deployment.

Image-goal planning first creates a representation tension. The planner needs a compact coordinate system in which a predicted future can be compared directly with a static goal image. A typed task configuration—such as position or joint angle—provides such a geometry, but is generally not a sufficient dynamical state: two histories can share the same configuration while differing in velocity, direction of motion, contact, or future controllability. Conversely, using an unrestricted visual latent

for both prediction and goal comparison mixes these dynamical distinctions with appearance variation that need not affect the terminal goal cost. A useful planning state must therefore be goal-comparable without discarding the history required to predict its evolution.

Offline learning creates a second, complementary gap. A recorded trajectory contains the future produced by the action that was actually executed at each history; it does not show what the same system state would have done under other actions. CEM, by contrast, evaluates hundreds of alternative sequences from the current history. Factual training therefore constrains the transition at the executed action but leaves the outcomes of alternative actions underdetermined. This gap is distinct from long-horizon error accumulation: limited action variation at a fixed state can leave controlled dynamics unidentified from the recorded data (Zhang et al., 2026b).

FIRM-WM addresses the two gaps with a factual–interventional recurrent state factorization. It represents the planning state as

$$s_t = (c_t, m_t), \quad c_t \in \mathcal{C}, \quad m_t \in \mathbb{R}^{128}, \quad (1)$$

where c_t is a typed physical configuration that can be compared with a static goal, and m_t is a learned dynamic fiber that retains the history-dependent information needed to predict future configurations. The transition updates both variables recursively, but the terminal goal cost compares only c_t . Configuration and fiber therefore serve distinct roles: the former is the variable used by the terminal goal cost, while the latter completes the predictive state.

The model is trained from two complementary sources. Broad factual sequences cover the state distribution. For each intervention anchor, we reset the environment and restore the same recorded values of all variables exposed through the environment’s state-setting interface before executing each alternative action sequence. We refer to the resulting trajectories as *common-reset intervention branches*. “Common reset” denotes equality of the restored interface variables across branches, not identity of the complete simulator state or internal solver memory. These trajectories directly supervise how alternative actions change the configuration and auxiliary dynamics. The distinction is important: related physical-grounding methods decode state from latents (Yan et al., 2026) or separate predictions under perturbed actions (Zeng et al., 2026), whereas our intervention targets are observed environment trajectories produced by actions executed from common resets. Figure 1 shows the resulting image-goal planning architecture, and Table 1 positions its supervision, planner, and active model size against representative pixel world models.

Our contributions are:

- We formulate the *factual–interventional gap* in reward-free latent planning: factual prediction constrains the recorded action, while CEM depends on alternative-action outcomes from the current state.
- We introduce a compact recurrent state factorization that separates a typed goal configuration from a learned dynamic fiber, preserving history-dependent dynamics while making candidate rollouts and terminal scores interpretable in task coordinates.
- We train one recurrent transition with broad factual trajectories and common-reset intervention branches, using a shared multi-horizon physical loss and no reward, success, ranking, or planner-generated label.

2 RELATED WORK

Latent world models for planning. PlaNet and subsequent model-based RL agents learn latent dynamics from pixels, but use reward prediction and task-specific behavior learning (Hafner et al., 2019; 2025; Hansen et al., 2024). Reward-free visual planners instead learn from offline trajectories and optimize new image goals at deployment. DINO-WM predicts frozen DINOv2 patch features (Oquab et al., 2024; Zhou et al., 2025); PLDM studies when latent planning is preferable to offline goal-conditioned RL (Sobal et al., 2025); and LeWM trains an end-to-end JEPA from pixels with a compact predictor and isotropic latent regularization (Assran et al., 2023; Maes et al., 2026). Fast-LeWM predicts action prefixes in parallel to reduce autoregressive rollout cost (Gao and Xu, 2026). These approaches typically ask one latent representation to support both action-conditioned prediction and terminal goal comparison; LeWM, for example, ranks candidates by squared distance

Table 1: Representative pixel world models.

Method	Pixel deployment	Reward-free WM	Common-reset branches	Planning	Active parameters
RC-aux (Li et al., 2026)	✓	✓	–	reachability-aware CEM	$\sim 15.7 \text{ M}^\dagger$
SAGE (Cheng et al., 2026)	✓	✓	–	subgoal prior + CEM	$\sim 49.3 \text{ M}^\dagger$
LeWM (Maes et al., 2026)	✓	✓	–	CEM	$\sim 15 \text{ M}$
Fast-LeWM (Gao and Xu, 2026)	✓	✓	–	CEM	$\sim 18 \text{ M}$
PSG-JEPA (Yan et al., 2026)	✓	✓	–	CEM / policy	NR
PhyLatent (Zeng et al., 2026)	✓	✓	–	CEM	LeWM backbone
FIRM-WM (ours)	✓	✓	✓	CEM	2.98–3.42 M

NR: not reported in a directly comparable active-deployment accounting. $\dagger$ Active-stack estimates include the reported $\sim 15 \text{ M}$ LeWM backbone plus the online RC-aux head (0.675M) or SAGE subgoal and action generators (34.31M).

between the predicted terminal embedding and the encoded goal (Maes et al., 2026). FIRM-WM instead uses a typed configuration for terminal goal comparison and a separate history-dependent fiber for prediction. Candidate rollouts and terminal scores are therefore directly interpretable in task coordinates rather than only through an unrestricted embedding distance.

Physical structure and controlled identifiability. Physical latent models have long sought coordinates that simplify control (Jaques et al., 2021). Recent work adds physical probes or auxiliary objectives to JEPA world models. PSG-JEPA grounds individual latents and latent changes in robot state (Yan et al., 2026). Concurrent PhyLatent targets invariance, physical identifiability, and predicted branch separation while removing its auxiliary heads at deployment (Zeng et al., 2026). These methods establish that global non-collapse does not imply control-relevant local geometry. Recent theory casts Gaussian anti-collapse as a distributional anchor whose residual orthogonal symmetry matches Euclidean planning, while warning that exact isotropy in an overcomplete latent can crumple the data manifold (Zhang et al., 2026c). Physical coordinates provide an interpretable goal geometry, but using them as the entire state can discard the dynamic context required for closed prediction. FIRM-WM therefore pairs a typed, goal-comparable configuration with an unrestricted dynamic fiber: the former defines the terminal cost, while the latter distinguishes histories with the same configuration but different futures. Weak conditional action excitation leaves those futures underdetermined for alternative actions (Zhang et al., 2026b); executed common-reset interventions supervise observed outcomes under sampled alternative actions.

Planning in learned latent spaces. Latent MPC can be improved through temporal hierarchy (Zhang et al., 2026a), learned subgoal proposals (Cheng et al., 2026), amortized inverse dynamics (Nguyen et al., 2026), or cost-ranking objectives (Chahe et al., 2026). These methods improve how candidate trajectories are proposed or ranked. FIRM-WM instead changes the state and transition supplied to the search: CEM compares predicted goal configurations, while the dynamic fiber carries information needed to produce those predictions under alternative actions. Our comparisons hold the CEM procedure and candidate budget fixed.

3 PROBLEM SETTING: REWARD-FREE IMAGE-GOAL PLANNING

We study visual control from offline trajectories without reward supervision. At deployment, the controller receives a goal image o^g and the causal history

$$h_t = (o_{t-L+1:t}, a_{t-L+1:t-1}),$$

which contains the last L images and the raw actions executed between them. The symbol h_t always denotes the currently available history: during training it is taken from a recorded trajectory, whereas during deployment it is formed from live observations and executed actions. To reach the goal, CEM evaluates a horizon- H sequence of macro actions $U = (u_0, \dots, u_{H-1})$ by rolling a learned world model forward, then executes the first macro action and replans from the next observation.

The central difficulty is a mismatch between what the offline data record and what the planner asks the model to predict. The factual dataset $\mathcal{D}_F$ contains image-action trajectories collected before training. At a history sampled from one of those trajectories, it provides the future following the action sequence that was actually recorded; we call this the *factual* continuation. During deployment, CEM instead proposes many candidate sequences from the current live history, predicts their futures

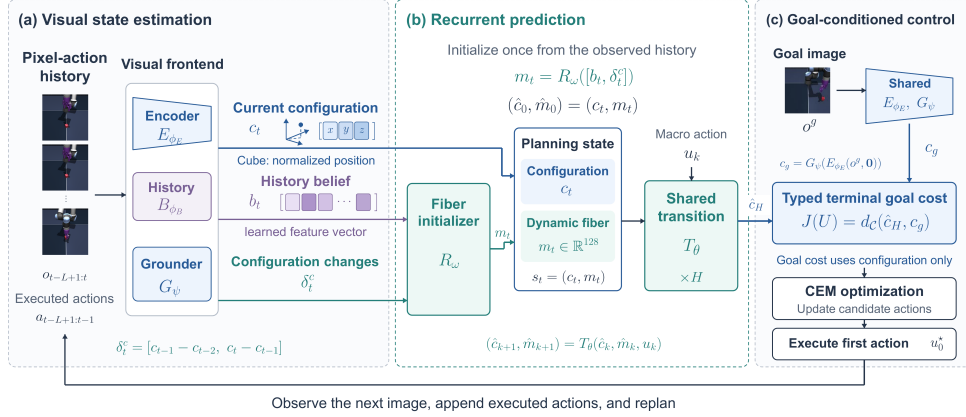


Figure 1: **FIRM-WM image-goal planning architecture.** The pixel-action history is mapped by the encoder, causal history module, and grounder to the current configuration c_t and belief b_t . The history-to-fiber initializer constructs m_t , giving the recurrent state $s_t = (c_t, m_t)$. For each candidate macro-action sequence, the single shared transition recursively updates configuration and fiber for H steps. The shared encoder and grounder map the goal image to c_g , and the typed terminal distance $d_C(\hat{c}_H, c_g)$ supplies the CEM objective $J(U)$. The controller executes the first optimized macro action, appends the next observation and executed action to the causal history, and replans. The fiber affects prediction but is excluded from the goal cost.

with the world model, and repeatedly updates its candidates toward lower predicted goal cost. Most of these candidates have no corresponding real outcome in $\mathcal{D}_F$ from exactly the same physical state. We call their predicted futures *counterfactual planner queries*. The recorded continuation therefore does not by itself determine the outcomes of the alternatives that CEM compares when choosing an action.

A world model for this setting must therefore satisfy three properties. Its predicted terminal state must contain a coordinate that can be compared directly with the goal image; its recurrent state must retain the history-dependent information needed to predict how that coordinate evolves; and its training signal must constrain outcomes under actions beyond the single factual continuation observed at each history. Section 4 instantiates these requirements with a configuration-fiber recurrent state and complementary factual and common-reset intervention rollouts.

4 FIRM-WM: PIXEL-BASED COUNTERFACTUAL WORLD MODELING

FIRM-WM realizes these requirements in one recurrent planning pipeline. Spatial tokens and a causal belief infer the current situation from pixels; a typed configuration is compared with the goal image by the terminal cost; and a compact dynamic fiber retains the additional history needed for prediction. The shared transition learns from broad factual rollouts together with common-reset intervention branches, then supplies counterfactual configuration rollouts to CEM. Figure 1 summarizes this image-goal planning architecture.

4.1 PIXEL TOKENS AND CAUSAL HISTORY BELIEF

A current image reveals scene geometry but not, by itself, how that scene is evolving. We preserve its spatial organization with a small set of local visual tokens,

$$p_t = E_{\phi_E}(o_t, \Delta o_t), \quad \Delta o_t = o_t - o_{t-1}. \quad (2)$$

Here, E_{ϕ_E} denotes the pixel frontend: a spatial tokenizer for TwoRoom, Push-T, and Cube, and a compact geometry encoder for Reacher. Its output encodes where task-relevant structures appear in the current frame. To recover motion information that is ambiguous in a single image, the causal history attention module fuses the recent token sequence with the actions executed between frames,

$$b_t = B_{\phi_B}(p_{t-L+1:t}, a_{t-L+1:t-1}). \quad (3)$$

Thus p_t describes the current visual arrangement, whereas b_t recovers motion evidence unavailable from one frame and summarizes the visual–action history needed to predict its evolution. The task-typed frontend architectures, token dimensions, and history lengths are reported in Appendix B; all tasks expose the same token–belief interface to the recurrent state below.

4.2 CONFIGURATION–FIBER RECURRENT DYNAMICS

The goal-comparable component of the recurrent state is a typed configuration inferred from a canonical encoding of the current image,

$$c_t = G_\psi(E_{\phi_E}(o_t, \mathbf{0})) \in \mathcal{C}. \quad (4)$$

For the spatial encoders, the zero second input removes the frame-difference channel from this configuration readout. Depending on the task, c_t contains position, periodic joint angles, or object pose components; Appendix B specifies each interface. The grounder G_ψ is supervised by physical configuration labels during training and shared between current and goal images. Its output therefore supplies the typed geometry in which terminal predictions are compared with the goal.

We train the pixel frontend and configuration grounder before the reported recurrent-dynamics refinement. The pixel frontend uses task-typed visual and configuration-preserving objectives, while the grounder regresses the physical configuration coordinates defined in Appendix B. The causal history module is then either frozen or jointly refined with the recurrent transition according to the task schedule. The history-to-fiber initializer and unified transition are trained recursively on factual and common-reset intervention rollouts. Appendix D reports each module’s initialization, objective, optimizer, update count, freezing rule, and checkpoint-selection criterion for every environment.

Configuration alone is not a sufficient dynamical state. Two histories can have the same c_t while differing in velocity, motion direction, contact mode, or other variables that change their action-conditioned futures. FIRM-WM retains this residual dynamic context in a compact fiber. A history-to-fiber encoder constructs the initial fiber state for each candidate rollout from the causal belief and recent changes in configuration,

$$\delta_t^c = [c_{t-1} - c_{t-2}, c_t - c_{t-1}], \quad m_t = R_\omega([b_t, \delta_t^c]) \in \mathbb{R}^{128}, \quad (5)$$

where $[\cdot, \cdot]$ denotes feature concatenation. The encoder R_ω is evaluated once at the current observed history to initialize m_t ; subsequent fiber states are produced recursively by the transition. For an angular coordinate ϑ , c_t stores $(\sin \vartheta, \cos \vartheta)$. Differences in δ_t^c are therefore taken between sine–cosine pairs and do not jump when the angle crosses $\pm\pi$. The resulting planning state $s_t = (c_t, m_t)$ separates two roles: c_t provides goal geometry, while m_t retains history-dependent information that is excluded from the goal cost but needed to predict the next recurrent state. For one candidate rollout, set $(\hat{c}_0, \hat{m}_0) = (c_t, m_t)$. The recurrent transition updates both components,

$$(\hat{c}_{k+1}, \hat{m}_{k+1}) = T_\theta(\hat{c}_k, \hat{m}_k, u_k), \quad k = 0, \dots, H-1, \quad (6)$$

where u_k is the k th macro action in the candidate sequence U . Across all tasks, T_θ remains one recurrent operator with one configuration head and one fiber head. TwoRoom, Reacher, and Cube use a compact residual-MLP hidden update. Push-T retains the same state, action, and output interface but adds an action-conditioned spatial residual inside the same hidden update to preserve contact geometry. Under our settings, CEM evaluates this operator over 15,000–45,000 candidate–step pairs per solve, without repeated token-level attention in the planning inner loop. The transition applies its increments to Euclidean position coordinates, sine–cosine angle coordinates, and the fiber, so both the predicted configuration and dynamic context evolve throughout the rollout. During training, an auxiliary decoder D_ρ maps $\hat{m}_k$ to an environment-typed dynamic target, $\hat{\chi}_k = D_\rho(\hat{m}_k)$, encouraging the fiber to retain variables such as velocity or contact. The decoder and target are discarded after training and never enter the goal cost. Network widths and task-typed residual details are given in Appendix B.

4.3 LEARNING FROM FACTUAL AND INTERVENTIONAL ROLLOUTS

Learning the transition requires two complementary kinds of evidence. Factual rollouts cover the broad range of states present in the offline trajectories, but provide little action variation at any one history. Common-reset interventions concentrate on fewer histories but reveal how alternative actions change their futures.

A factual record is a rollout $\xi^F = (h_i, U_i^F, c_{i,0:H}, \chi_{i,0:H})$ extracted from an offline trajectory. Here $c_{i,k}$ is the goal-comparable configuration defined above, while $\chi_{i,k}$ is an environment-typed auxiliary dynamics vector, such as velocity, displacement, contact, or the remaining robot state. The factual dataset $\mathcal{D}_F$ contains these records across the state coverage of the offline trajectories.

To collect common-reset intervention data, we select a time index i from a training episode and save two items: the observation–action history h_i and the recorded reset condition x_i^{sim} . Here, x_i^{sim} contains only the physical variables exposed by the environment’s state-setting interface. For each alternative macro-action sequence U_{ij} , we first reset the environment, restore x_i^{sim} , execute U_{ij} , and record the resulting configuration and auxiliary-dynamics trajectories. Repeating this procedure for K_i action sequences gives

$$\mathcal{I}_i = \{\xi_{ij}^I = (h_i, U_{ij}, c_{ij,0:H}, \chi_{ij,0:H})\}_{j=1}^{K_i}. \quad (7)$$

These branches begin from the same restored interface variables and differ in their action sequences. The symbol ξ_{ij}^I denotes one resulting training record, and $\mathcal{D}_I$ is the collection of these records. Thus $\mathcal{D}_F$ provides broad state coverage, while $\mathcal{D}_I$ provides observed futures for alternative actions under common reset conditions. Appendix D specifies the reset variables, anchor counts, action sampling, and branch counts for every task; Appendix B gives the task-specific contents of χ and the raw-to-macro timing. A factual-suffix reset-replay audit over 1,210 eligible training anchors keeps the P95 task-normalized configuration discrepancy below 0.15 success-tolerance units at every macro horizon (Table 10). The only auxiliary decoder in this stage is D_ρ , which maps m_k to χ_k during training and is absent from the planner.

For environment e , recurrent stage r activates sources $\mathcal{A}_{e,r} \subseteq \{F, I\}$ and applies $N_{e,r}$ updates to parameters carried from the preceding stage. Algorithm 1 and Appendix D give the complete module and source schedules.

For either source $\sigma \in \{F, I\}$, the rollout loss has three functional parts,

$$\ell_\sigma(\mathcal{B}) = \underbrace{\ell_{\sigma,\text{cfg}}(\mathcal{B})}_{\text{future configuration}} + \underbrace{\ell_{\sigma,\text{disp}}(\mathcal{B})}_{\text{configuration displacement}} + \underbrace{\ell_{\sigma,\text{dyn}}(\mathcal{B})}_{\text{fiber dynamics}}. \quad (8)$$

The configuration term supervises the absolute future $c_{1:H}$ using squared Euclidean error for position coordinates and squared distance between sine–cosine pairs for angles. The displacement term matches changes between configurations, so the loss measures the effect of the executed action rather than allowing static scene content to dominate. The dynamics term supervises $D_\rho(\hat{m}_k)$ with the training-only targets χ_k . Appendix E defines the displacement operator Λ_κ and gives the exact tail reductions, initial-fiber term, and component weights used by each task and data source. In the final recurrent stage of every environment, both sources update one transition through the joint objective

$$\mathcal{L}_{\text{FIRM}} = \mathbb{E}_{\mathcal{B}_F \sim \text{Batch}(\mathcal{D}_F)}[\ell_F(\mathcal{B}_F)] + \mathbb{E}_{\mathcal{B}_I \sim \text{Batch}(\mathcal{D}_I)}[\ell_I(\mathcal{B}_I)], \quad (9)$$

where $\text{Batch}(\mathcal{D})$ denotes a minibatch drawn from dataset $\mathcal{D}$. The full-pipeline factual-only and intervention-only variants in Appendix F restrict every task-training stage to $\mathcal{D}_F$ or $\mathcal{D}_I$, respectively, while retaining the architecture, initialization scheme, optimization schedule, and paired evaluation protocol.

Equation (9) therefore joins state coverage with action coverage at common reset conditions: factual data teach how the system evolves along recorded trajectories, while interventions teach how alternative actions change that evolution after restoring the same interface variables. Earlier recurrent stages use the active source sets listed in Appendix D. The complete recurrent-transition update count is $N_{\text{rec}}^{(e)} = \sum_{r=1}^{R_e} N_{e,r}$. The intervention records in Equation (7) are constructed before recurrent training. Simulator resets therefore occur only during data collection; Algorithm 1 operates on completed factual and intervention records.

4.4 REWARD-FREE LATENT MPC

First, the goal image is encoded by the same pixel encoder and grounder as the current observation,

$$p^g = E_{\phi_E}(o^g, \mathbf{0}), \quad c_g = G_\psi(p^g). \quad (10)$$

Algorithm 1 Complete recurrent-transition training across environment-specific stages

Require: For environment e : completed data, source sets $\mathcal{A}_{e,r}$, update counts $N_{e,r}$, fixed input modules, recurrent initializer, AdamW optimizer, and horizon H

Ensure: Deployment modules $(E_{\phi_E}, B_{\phi_B}, G_{\psi}, R_{\omega}, T_{\theta})$

```
1:  $\Theta_{\text{train}} := (\omega, \theta, \rho)$ 
2: for  $r = 1, \dots, R_e$  do
3:   Load fixed  $(E_{\phi_E}, B_{\phi_B}, G_{\psi})$  and  $\{\mathcal{D}_{\sigma} : \sigma \in \mathcal{A}_{e,r}\}$ 
4:   If needed, add zero-initialized input weights without changing existing weights
5:   for  $n = 1, \dots, N_{e,r}$  do
6:     for  $\sigma \in \mathcal{A}_{e,r}$  do
7:       Sample  $\mathcal{B}_{\sigma} \sim \text{Batch}(\mathcal{D}_{\sigma})$ ; infer  $(\hat{c}_0^{\sigma}, \hat{m}_0^{\sigma})$ ; roll out  $T_{\theta}$  for  $H$  steps
8:     end for
9:      $\mathcal{L} \leftarrow \sum_{\sigma \in \mathcal{A}_{e,r}} \ell_{\sigma}(\mathcal{B}_{\sigma})$ 
10:     $\Theta_{\text{train}} \leftarrow \text{AdamWStep}(\Theta_{\text{train}}, \nabla_{\Theta_{\text{train}}} \mathcal{L})$ 
11:   end for
12: end for
13: Discard  $D_{\rho}$  and return  $(E_{\phi_E}, B_{\phi_B}, G_{\psi}, R_{\omega}, T_{\theta})$ 
```

Second, Equation (6) recursively predicts the terminal configuration $\hat{c}_H(U)$ for every candidate macro-action sequence U . Third, the planner scores that candidate with the task-typed configuration distance

$$J(U; h_t, o^g) = d_{\mathcal{C}}(\hat{c}_H(U), c_g), \quad (11)$$

where $d_{\mathcal{C}}$ respects Euclidean and periodic components of the task configuration. The fiber affects the predicted future but is not itself compared with the goal. Unlike a terminal distance over an unrestricted learned embedding, this objective makes candidate ranking directly interpretable: its score is expressed through named quantities such as position and joint-angle error, with the exact decomposition given in Appendix B. Finally, CEM minimizes Equation (11); the controller executes the first optimized macro action, observes the next image, and replans.

5 EXPERIMENTS AND RESULTS

5.1 EXPERIMENTAL SETUP

We evaluate image-goal control on TwoRoom navigation, two-joint Reacher, Push-T planar manipulation, and OGBench-Cube manipulation (Park et al., 2025). Each environment uses fixed training and validation splits. The training split supplies factual trajectories and intervention anchors, the validation split supplies held-out representation and rollout metrics, and final paired control is measured on a fixed set of 100 trials.

FIRM-WM and LeWM use the same task-specific CEM protocol and the same 100 paired trials in each environment. Appendix B reports the complete candidate, iteration, elite, horizon, action-block, and extraction settings. The Cube appendix additionally reports the public seed-42 50-episode reproduction.

Training variance. To assess training stability, we retrain the complete FIRM-WM pipeline with three independent random seeds. Each run initializes all trainable stages independently while holding the training split, validation split, intervention data, training recipe, and paired evaluation trials fixed.

Baselines and access. LeWM is evaluated with its released checkpoint and solver (Maes et al., 2026). FIRM-WM uses physical states to supervise the grounder and common resets to collect intervention branches. Both methods receive pixels, past actions, and a goal image at deployment.

Metrics. We report binary task success over 100 paired episodes and the sample mean and standard deviation across three independently trained full-pipeline models.

5.2 GOAL-CONDITIONED CONTROL

Table 2 places FIRM-WM in the broader published landscape, while Figure 2 reports our matched checkpoint comparison. The individual FIRM-WM success rates for Seeds 1–3 are 99%, 100%, and

Table 2: Planning success rate (%) across four continuous-control environments. FIRM-WM reports mean $\pm$ sample s.d. over three independent full-pipeline training seeds, each evaluated on the same 100 paired trials. All comparison rows are reproduced from Table 1 of Zhao et al. (2026): its LeWM and Sub-JEPA entries average six evaluation seeds with 50 trajectories per seed, whereas its PLDM and DINO-WM entries are transcribed from Maes et al. (2026). Bold highlights the leading result(s) in each column; on the near-saturated TwoRoom task, both DINO-WM’s 100.00 and FIRM-WM’s 99.0 ± 1.0 are highlighted.

Method	TwoRoom	Reacher	Push-T	OGBench-Cube
PLDM (Sobal et al., 2025)	97.00	78.00	78.00	65.00
DINO-WM (Zhou et al., 2025)	100.00	79.00	74.00	86.00
LeWM (Maes et al., 2026)	84.33 ± 4.23	82.67 ± 4.42	84.67 ± 6.53	67.33 ± 5.01
Sub-JEPA (Zhao et al., 2026)	95.00 ± 2.76	84.00 ± 4.00	89.00 ± 5.33	76.33 ± 5.99
FIRM-WM (ours)	99.0 ± 1.0	92.7 ± 2.1	75.7 ± 4.7	88.0 ± 3.0

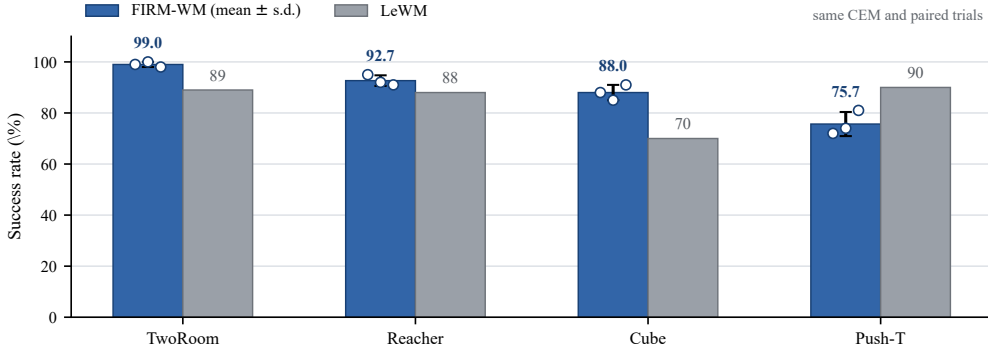


Figure 2: **Control results with optimization variability.** Bars show the mean of three independently trained FIRM-WM seeds; error bars are sample standard deviations and circles are individual seeds. LeWM uses its fixed checkpoint on the same paired evaluation trials and matched CEM budget. FIRM improves all three seeds on TwoRoom, Reacher, and Cube; Push-T exposes the persistent-contact boundary.

98% on TwoRoom; 95%, 92%, and 91% on Reacher; and 88%, 85%, and 91% on Cube. The released LeWM checkpoint reaches 89%, 88%, and 70% on the same respective paired trials. Thus every FIRM-WM seed exceeds the matched checkpoint in these three environments; its mean success rate is 99.0% on TwoRoom, 92.7% on Reacher, and 88.0% on Cube. The corresponding Push-T seed results are 72%, 74%, and 81%.

Push-T yields $75.7 \pm 4.7\%$ versus 90.0% for LeWM, with the same direction across all three independently trained models. Its sustained object contact couples pose, velocity, and solver mode over the planning horizon, one regime not closed by the present compact deterministic transition.

5.3 COMPACTNESS AND PLANNING EFFICIENCY

Table 3: Active deployment size and recorded planner-loop time on the same evaluation host. FIRM seconds are mean $\pm$ sample s.d. over three full-pipeline seeds; the LeWM checkpoint is fixed. Speedup averages the three per-seed ratios.

Environment	FIRM active	FIRM sec./ep.	LeWM sec./ep.	Speedup	Matched CEM budget
TwoRoom	3.41M	0.416 ± 0.056	3.224	$7.83 \times$	300 / 30 it. / top-30 / $H=5$
Reacher	3.16M	0.504 ± 0.149	5.550	$11.60 \times$	300 / 30 it. / top-30 / $H=5$
Push-T	2.98M	1.860 ± 0.243	3.926	$2.13 \times$	300 / 30 it. / top-30 / $H=5$
OGBench-Cube	3.42M	0.635 ± 0.118	2.389	$3.84 \times$	300 / 10 it. / top-30 / $H=5$
LeWM reported model size	~ 15 M active parameters; time normalized above within each task				identical per row

Table 4: Full-chain training-source ablations. Values are success (%), mean $\pm$ sample s.d. over three independently initialized training runs. $\dagger$ marks Reacher-only single-seed diagnostics; LeWM is the fixed paired baseline. All entries use released-original CEM.

Training condition	Reacher	Cube
Factual only	89.3 ± 1.5	79.7 ± 1.5
Intervention only	71.7 ± 4.6	82.0 ± 1.7
Factual + intervention (FIRM)	92.7 ± 2.1	88.0 ± 3.0
No history fiber †	68.0	–
Mismatched outcomes †	16.0	–
LeWM (fixed paired baseline)	88.0	70.0

FIRM’s active model is $4.4\text{--}5.0\times$ smaller than the approximately 15M-parameter count reported for LeWM. Among the multi-environment reward-free visual world models in Table 1 with disclosed comparable active counts, FIRM has the smallest active deployment model.

The smaller recurrent state also lowers the cost of evaluating CEM candidates. Under the matched paired evaluator, the recorded planner loop is $2.13\text{--}11.60\times$ faster across TwoRoom, Reacher, Push-T, and Cube, measured over current/goal encoding, recurrent rollout, CEM population updates, and action selection.

5.4 MECHANISM ABLATION

Across Reacher and Cube, the combined factual–interventional pipeline outperforms the best single-source variant in every independently initialized full-pipeline run (Table 4): $92.7 \pm 2.1\%$ versus $89.3 \pm 1.5\%$ on Reacher, and $88.0 \pm 3.0\%$ versus $82.0 \pm 1.7\%$ on Cube. Two Reacher-only single-seed diagnostics separately isolate the effects of removing the history fiber (68.0%) and mismatching intervention actions and outcomes (16.0%). Together, the full-chain ablations support complementary factual and common-reset supervision, while the diagnostics support history-dependent state and correct action–outcome correspondence. Appendix F and Figure 3 give the protocols and per-run results.

6 LIMITATIONS AND DISCUSSION

Training-time supervision and intervention access. FIRM uses physical configuration labels to train G_ψ and environment state-setting interfaces to collect alternative-action trajectories from common resets. The former is supervision for a learned module; the latter is used only during data collection. Neither is available to the deployed controller, which uses images, action history, and a goal image.

Intervention support and configuration. Common-reset interventions calibrate the sampled anchors and actions, while the typed configuration assumes that goal variables and their topology are known. Broader intervention acquisition and learned goal quotients are natural extensions when either support is unavailable.

Persistent contact. On Push-T, FIRM averages $75.7 \pm 4.7\%$ versus 90.0% for LeWM. Sustained contact couples geometry, velocity, and solver mode over multiple CEM steps; the result is consistent with a compact deterministic transition being insufficient for this multimodal regime. Richer conditional dynamics are the direct next extension.

7 CONCLUSION

Reward-free pixel planning requires a state that is both goal-comparable and dynamically sufficient, together with a transition calibrated for the alternative actions queried by CEM. FIRM-WM supplies these properties through a recurrent factorization into a goal-comparable configuration and a history-dependent dynamic fiber, trained on factual trajectories and common-reset interventions. Across three

independently initialized models, it improves matched LeWM control on TwoRoom, Reacher, and Cube while using fewer active parameters and less recorded planner-loop time. Push-T reverses the control advantage and locates the remaining challenge in persistent-contact dynamics. Taken together, the results support state factorization and common-reset interventional calibration as practical design principles for compact reward-free pixel planning.

REFERENCES

- M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15619–15629, 2023. URL https://openaccess.thecvf.com/content/CVPR2023/html/Assran_Self-Supervised_Learning_From_Images_With_a_Joint-Embedding_Predictive_Architecture_CVPR_2023_paper.html.
- A. Chahe, S. Cai, and L. Zhou. Latent world models with monotone planning costs for image-goal navigation. *arXiv preprint arXiv:2608.09073*, 2026. URL <https://arxiv.org/abs/2608.09073>.
- L. Cheng, Q. Zhang, and Y. Wang. SAGE: Subgoal-conditioned action generation for latent world model planning. *arXiv preprint arXiv:2607.17973*, 2026. URL <https://arxiv.org/abs/2607.17973>.
- Y. Gao and X. Xu. Fast leworldmodel. *arXiv preprint arXiv:2606.26217*, 2026. URL <https://arxiv.org/abs/2606.26217>.
- D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2555–2565. PMLR, 2019. URL <https://proceedings.mlr.press/v97/hafner19a.html>.
- D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025. doi: 10.1038/s41586-025-08744-2. URL <https://www.nature.com/articles/s41586-025-08744-2>.
- N. Hansen, H. Su, and X. Wang. TD-MPC2: Scalable, robust world models for continuous control. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=Oxh5CstDJU>.
- M. Jaques, M. Burke, and T. M. Hospedales. NewtonianVAE: Proportional control and goal identification from pixels via physical latent spaces. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4454–4463, 2021. URL https://openaccess.thecvf.com/content/CVPR2021/html/Jaques_NewtonianVAE_Proportional_Control_and_Goal_Identification_From_Pixels_via_Physical_CVPR_2021_paper.html.
- W. Li, G. Li, K. Maeda, T. Ogawa, and M. Haseyama. Predictive but not plannable: RC-aux for latent world models. *arXiv preprint arXiv:2605.07278*, 2026. URL <https://arxiv.org/abs/2605.07278>.
- L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026. URL <https://arxiv.org/abs/2603.19312>.
- H. Nguyen, X. Xu, and X. Huang. Latent geometry beyond search: Amortizing planning in world models. *arXiv preprint arXiv:2605.08732*, 2026. URL <https://arxiv.org/abs/2605.08732>.
- M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=a68SUt6zFt>.

- S. Park, K. Frans, B. Eysenbach, and S. Levine. OGBench: Benchmarking offline goal-conditioned reinforcement learning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=M992mjgKzI>.
- R. Y. Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999. doi: 10.1023/A:1010091220143.
- U. Sobal, W. Zhang, K. Cho, R. Balestriero, T. G. J. Rudner, and Y. LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi: 10.52202/085713-1465. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/3e7cf447f21cd11c846463affefce665-Abstract-Conference.html.
- H. Yan, J. Zhu, M. Jia, R. Yin, J. He, Z. Zhong, J. Li, J. Lu, H. Li, T. Zhang, J. Chen, W. Song, W. Chen, Y. Gao, and H. Li. Is forward prediction enough? physical state grounding for JEPA world models. *arXiv preprint arXiv:2608.06799*, 2026. URL <https://arxiv.org/abs/2608.06799>.
- X. Zeng, H. Ren, and Z. Song. Phylatent: Learning dynamics-relevant representations for JEPA world models. *arXiv preprint arXiv:2608.05720*, 2026. URL <https://arxiv.org/abs/2608.05720>.
- W. Zhang, B. Terver, A. Zholus, S. Chitnis, H. Sutaria, M. Assran, R. Balestriero, A. Bar, A. Bardes, Y. LeCun, and N. Ballas. Hierarchical planning with latent world models. *arXiv preprint arXiv:2604.03208*, 2026a. URL <https://arxiv.org/abs/2604.03208>.
- X. Zhang, Y. Guan, B. Zhang, H. Li, Y.-Q. Zhang, and S. E. Li. On the identifiability of controlled world models. *arXiv preprint arXiv:2607.22430*, 2026b. URL <https://arxiv.org/abs/2607.22430>.
- Y.-J. Zhang, C.-W. Liang, T.-Y. Zhang, J. Ding, Y.-L. Wu, A.-B. Li, W.-C. Su, Saifullah, H.-Y. An, and M.-J.-S. Wang. Identifiability and latent geometry of joint-embedding predictive world models. *SSRN Electronic Journal*, June 2026c. doi: 10.2139/ssrn.7158998. URL <https://ssrn.com/abstract=7158998>.
- K. Zhao, D. Nie, Y. Lin, Z. Luo, Y. Gu, D.-P. Fan, and D. Zeng. Sub-JEPA: Subspace gaussian regularization for stable end-to-end world models. *arXiv preprint arXiv:2605.09241*, 2026. URL <https://arxiv.org/abs/2605.09241>.
- G. Zhou, H. Pan, Y. LeCun, and L. Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 79115–79135. PMLR, 2025. URL <https://proceedings.mlr.press/v267/zhou25t.html>.

A NOTATION REFERENCE

Table 5: Notation used throughout the paper. Raw environment time t and local macro index k are distinct. The symbol χ denotes training-only auxiliary dynamics, while $d_{\mathcal{C}}$ denotes the deployment goal cost.

Group	Symbol	Meaning
Environment	t	Raw environment time index.
	o_t	Pixel observation at raw time t .
	$a_t \in \mathcal{A}$	Raw control applied at time t .
	h_t	Causal observation–action history available at time t .
	o^g	Goal image.
Macro rollout	S	Number of raw controls in one macro action.
	k	Local macro-step index within a predicted rollout.
	H	Planning horizon in macro steps.
	u_k	Macro action applied at rollout step k .
	U	Candidate sequence of H macro actions.
Encoded state	p_t	Spatial visual tokens extracted from o_t .
	b_t	Causal history belief formed from recent tokens and actions.
	$s_t = (c_t, m_t)$	State used by the recurrent transition.
	c_t	Typed, goal-comparable configuration.
	δ_t^c	Configuration innovation used to initialize the fiber.
	m_t	History-dependent dynamic fiber.
Learned modules	χ_t	Environment-typed dynamic target decoded from m_t during training only.
	E_{ϕ_E}	Pixel encoder that produces spatial tokens.
	B_{ϕ_B}	Causal history-belief module.
	G_{ψ}	Configuration grounder.
	R_{ω}	History-to-fiber initializer.
	T_{θ}	Recurrent action-conditioned transition.
Supervision	D_{ρ}	Training-only auxiliary dynamics decoder.
	x_i^{sim}	Recorded environment state at intervention anchor i .
	$\mathcal{D}_{\text{F}}$	Factual rollout dataset.
	$\mathcal{D}_{\text{I}}$	Common-reset intervention rollout dataset.
	$\mathcal{E}_{ij}^{\text{I}}$	Intervention branch j from anchor i .
	K_i	Number of intervention branches at anchor i .
	$\sigma \in \{\text{F}, \text{I}\}$	Data-source index: factual or interventional.
	$\mathcal{A}_{e,r}$	Active data sources in recurrent stage r of environment e .
	R_e	Number of recurrent stages for environment e .
	$N_{e,r}$	Number of updates in recurrent stage r of environment e .
Loss	$N_{\text{rec}}^{(e)}$	Total recurrent-transition updates for environment e .
	$\mathcal{B}$	Training minibatch.
	Λ_{κ}	Topology-aware configuration displacement map.
	$q(k)$	Predecessor index used by the recursive loss.
	c_k	Anchored configuration target at macro step k .
	$\mathcal{E}(\mathcal{B})$	Multiset of per-example tail errors in minibatch $\mathcal{B}$.
Planning	ℓ_{σ}	Training loss for source σ .
	$d_{\mathcal{C}}$	Squared terminal distance in configuration space.
	J	Objective assigned to a candidate action sequence.

B TASK INTERFACES AND MODEL GEOMETRY

The recurrent transition evolves in macro time. If one macro action contains S raw controls, then

$$u_k = (a_{t+Sk}, \dots, a_{t+S(k+1)-1}) \in \mathcal{A}^S, \quad U = (u_0, \dots, u_{H-1}). \quad (12)$$

All experiments use $S = 5$ and $H = 5$, so each candidate sequence spans 25 raw controls. Table 8 gives the remaining CEM contract.

For configurations with $N_{\circ}$ periodic scalar variables, write

$$c = (e(\vartheta_1), \dots, e(\vartheta_{N_{\circ}}), c^{\text{euc}}), \quad e(\vartheta) = (\sin \vartheta, \cos \vartheta), \quad (13)$$

where c^{euc} collects the non-periodic Euclidean coordinates of the task configuration. We then define

$$d_{\mathcal{C}}(c, c') = \|c^{\text{euc}} - c'^{\text{euc}}\|_2^2 + \sum_{\ell=1}^{N_{\circ}} \|e(\vartheta_{\ell}) - e(\vartheta'_{\ell})\|_2^2. \quad (14)$$

The periodic contribution in Equation (14) is the squared chord distance between two unit-circle encodings. Let $\delta_{\ell} = \text{wrap}(\vartheta_{\ell} - \vartheta'_{\ell}) \in [-\pi, \pi)$. Then $\|e(\vartheta_{\ell}) - e(\vartheta'_{\ell})\|_2^2 = 4 \sin^2(\delta_{\ell}/2)$, so equivalent

Table 6: Typed interfaces used by the shared configuration–fiber transition. History length is measured in observations; every macro action contains five raw environment controls.

Environment	$\dim(c)$	Goal-comparable configuration	Periodic pairs	$\dim(\chi)$	History frames	$\dim(u_k)$
TwoRoom	2	agent position	0	2	3	10
Reacher	4	sine/cosine of two joint angles	2	2	3	10
Push-T	6	block orientation and agent/block positions	1	5	3	10
OGBench-Cube	3	fixed affine-normalized block position in 3-D	0	25	3	25

Table 7: Environment-specific configuration and terminal-cost instantiations. Here p^a denotes agent position, p^b denotes block position, and a subscript g denotes the value inferred from the goal image. One periodic scalar is stored as one unit-circle pair $e(\vartheta) = (\sin \vartheta, \cos \vartheta)$; it does not introduce a second physical degree of freedom.

Environment	Configuration c	N_o	Terminal cost
TwoRoom	$c = p^a = (x^a, y^a)$	0	$\ p^a - p_g^a\ _2^2$
Reacher	$c = (e(\vartheta_1), e(\vartheta_2))$	2	$\sum_{j=1}^2 \ e(\vartheta_j) - e(\vartheta_{g,j})\ _2^2$
Push-T	$c = (e(\vartheta^b), p^a, p^b)$	1	$\max\{r_\vartheta, r_p\}^2$, where $r_\vartheta = \text{wrap}(\vartheta^b - \vartheta_g^b) /\tau_\vartheta$ and $r_p = \ \Delta p\ _2/\tau_p$ (task-normalized)
OGBench-Cube	fixed affine-normalized 3-D block position	0	$\ c - c_g\ _2^2$

angles on opposite sides of the $\pm\pi$ boundary remain close. TwoRoom and Cube have no periodic coordinates; the term is applied to both Reacher joints and to the Push-T block orientation.

Push-T uses the task’s fixed orientation and position tolerances to put its two error types on a common scale. Define the concatenated position error by $\Delta p = [(p^a - p_g^a)^\top, (p^b - p_g^b)^\top]^\top$. The normalized errors are $r_\vartheta = |\text{wrap}(\vartheta^b - \vartheta_g^b)|/(\pi/9)$ for block orientation and $r_p = \|\Delta p\|_2/(20/256)$ for agent–block position in normalized image coordinates. Its terminal cost $\max\{r_\vartheta, r_p\}^2$ is small only when both errors approach their respective tolerances. The other three tasks use the corresponding instance of Equation (14); for TwoRoom and Cube, the periodic sum is empty. The grounder and recurrent transition normalize every predicted sine–cosine block to unit length before the cost is evaluated.

The dynamic target is used only to shape the recurrent fiber. It contains configuration velocity or the remaining task state after removing the static goal configuration. In Cube, for example, the target includes robot joint position and velocity, end-effector pose, gripper state, contact indicator, and block orientation, while the terminal cost uses only affine-normalized block position. These simulator quantities supervise D_ρ during training but are not read by the controller. At deployment, the current image produces $c_t = G_\psi(E_{\phi_E}(o_t, \mathbf{0}))$, the goal image produces $c_g = G_\psi(E_{\phi_E}(o^g, \mathbf{0}))$, and CEM compares the recursively predicted $\hat{c}_H$ with c_g . Configuration is therefore inferred from pixels rather than measured from the simulator.

Concretely, TwoRoom uses normalized agent position for c_t and its finite displacement for χ_t . Reacher maps both joint angles to sine–cosine pairs and uses the two joint velocities as χ_t . The Push-T interface concatenates block-angle sine–cosine with normalized agent and block positions; its five-dimensional auxiliary target is the macro-step change in agent position, block position, and wrapped block angle. Cube uses a fixed affine-normalized three-dimensional block position for c_t and a 25-dimensional auxiliary target χ_t : six arm joint positions, six joint velocities, normalized 3-D end-effector position, end-effector yaw as a sine–cosine pair, gripper opening, gripper contact, and a sign-invariant 6-D block-rotation chart. These configuration coordinates use task-fixed transforms rather than data-estimated standardizers. Separately, model-side action statistics and, where used, auxiliary-dynamics statistics are estimated from the training split and frozen in the checkpoint; deployment reads those stored constants and does not query simulator state.

The spatial tokenizer used in TwoRoom and Cube has four convolutional stages with channels 32, 64, 96, 64, GroupNorm and GELU activations, and a learned 7×7 positional embedding. Push-T uses the same channel backbone with a 14×14 typed spatial output and a grid-preserving grounder. Training-only decoders and change-support heads are excluded from active deployment counts.

Reacher uses a compact geometry-oriented pixel encoder and a causal history filter. Across all tasks, FIRM-WM uses three observations, a 128-dimensional recurrent fiber, a 384-dimensional transition hidden state, one recurrent transition, five macro rollout steps, and a typed configuration update. The Push-T transition places an action-conditioned spatial residual inside the shared hidden update; it does not change the recurrent state, output heads, or planner interface.

Table 8: Released-original-CEM evaluation contract. Candidate action coordinates use the train-split standardizer stored in each checkpoint, search Gaussians are unbounded, and the deployed sequence is the final elite mean. No row changes the solver between FIRM-WM and LeWM.

Environment	Candidates	Iterations	Elites	Macro horizon	Raw steps / macro	Goal offset
TwoRoom	300	30	30	5	5	25
Reacher	300	30	30	5	5	25
Push-T	300	30	30	5	5	25
OGBench-Cube	300	10	30	5	5	25

C MODULE-WISE PARAMETER ACCOUNTING

Table 9: Active deployment parameters by module. Pixel + history frontend combines the deployed visual encoder/tokenizer and causal history module. Training-only reconstruction and change-support heads are omitted.

Environment	Pixel + history frontend	Configuration grounder	Recurrent transition	Active total
TwoRoom	0.374M	1.613M	1.424M	3.411M
Reacher	0.992M	0.828M	1.343M	3.162M
Push-T	0.449M	0.157M	2.371M	2.977M
Cube	0.376M	1.613M	1.434M	3.424M

Totals are direct sums of active modules and are rounded only for display.

D TRAINING AND EVALUATION DISCLOSURE

Data sets. The offline training and validation splits contain 9,000/1,000 episodes for TwoRoom, Reacher, and Cube, and 16,685/2,000 episodes for Push-T. Table 15 gives the factual rollout segments and intervention data used by the final joint recurrent stage. The same raw intervention records may be shared across independent training seeds, but each seed encodes them with its independently trained frontend and learns a separate transition.

Anchor and branch collection. An anchor is one eligible time index sampled from a training episode; anchors use distinct source episodes. The index must leave enough preceding observations to form the three-frame history h_i and enough remaining steps for a 25-control rollout. For TwoRoom, Push-T, and Cube, one branch replays the recorded 25-action suffix and the remaining branches use Gaussian action proposals clipped to the action bounds. The proposal statistics come from the training split for TwoRoom and Cube and from the complete offline action table for Push-T. Reacher samples all 16 branches uniformly from its action bounds. Every branch runs for 25 raw controls and records targets after controls 5, 10, 15, 20, and 25, giving $H = 5$ macro steps. Anchor selection and branch sampling do not use reward, success, goal distance, or planner rank.

Common-reset intervention collection. The collector does not serialize the simulator process. Before each intervention branch, we reset the environment and restore the same recorded values x_i^{sim} of the variables accepted by the task’s state-setting interface. For TwoRoom this condition is the two-dimensional agent position. Reacher restores the two joint positions and velocities, sets the target geometry, and updates the derived physics state. Push-T installs its seven-dimensional public state: agent position, block position and orientation, and agent velocity; other simulator variables take the environment’s reset values. Cube passes the 21-dimensional MuJoCo position and 20-dimensional velocity vectors through the released reset interface and sets the target block pose; the

solver warm-start state is reinitialized. Thus “common reset” means that the state variables exposed by the released environment interface are set to identical values for all branches at an anchor, not that the complete simulator memory image is saved and restored.

Table 10: Common-reset factual-suffix replay on training anchors. Errors are configuration discrepancies in task success-tolerance units after 25 controls ($H=5$); “Max” is over all five macro horizons. The fixed gate requires $P95 \leq 0.25$ at every horizon and $\text{Max} \leq 1.0$. This tests restored-interface fidelity, not identity of hidden simulator or solver state.

Environment	Anchors	H5 mean	H5 P95	Max	Gate
TwoRoom	128	0	0	0	Pass
Reacher	442	4.52×10^{-15}	1.75×10^{-14}	3.55×10^{-14}	Pass
Push-T	512	0.0613	0.1424	0.9548	Pass
OGBench-Cube	128	0.00352	0.01035	0.06111	Pass

Reacher includes the 442 of 512 prespecified anchors with a complete 25-control factual suffix; the other 70 are excluded solely by episode length and are not replaced.

Training protocol. Each seed independently initializes and trains the task-specific frontend, configuration grounder, and recurrent transition under the same fixed data split, intervention collections, update schedule, optimizer settings, and final-checkpoint rule. Algorithm 1 covers all updates to the recurrent modules. Table 12 gives the complete sequence. The frontend and grounder are fixed within each recurrent stage but may be refined between stages; recurrent parameters are carried forward. If a stage enlarges the recurrent input, as in Push-T, existing recurrent weights are retained and weights on the new input coordinates are initialized to zero.

For Push-T, the task-typed Tokenizer and Grounder are refined for 500 and 2,000 updates, respectively, before the final 2,000-update unified-transition refinement.

Planning. All main results use 300 CEM samples, top 30 elites, five macro steps, receding horizon five, and five raw environment steps per macro action. Search occurs in action coordinates standardized by the training-split mean and scale stored in the checkpoint, is unbounded, and returns the final elite mean. TwoRoom, Reacher, and Push-T use 30 CEM iterations; Cube follows the 10-iteration LeWM protocol (Maes et al., 2026).

E COMPLETE SUPERVISION AND TRAINING PROCEDURE

For either a factual rollout segment or an intervention branch, the target at macro index k is read after exactly $S_k = 5k$ raw simulator controls. Define the topology-aware relative-configuration map

$$\Gamma_C(c', c) = (c'^{\text{euc}} - c^{\text{euc}}, e(\vartheta'_1 - \vartheta_1), \dots, e(\vartheta'_{N_o} - \vartheta_{N_o})), \quad (15)$$

with $e(\vartheta) = (\sin \vartheta, \cos \vartheta)$. To represent the change from a reference configuration c to a later configuration c' , define the displacement operator

$$\Lambda_\kappa(c', c) = \begin{cases} c' - c, & \kappa = \text{emb}, \\ \Gamma_C(c', c), & \kappa = \text{rel}. \end{cases} \quad (16)$$

The map Λ_κ is deterministic rather than learned. For $\kappa = \text{emb}$, it subtracts the stored configuration coordinates directly. For $\kappa = \text{rel}$, Γ_C subtracts the Euclidean coordinates and composes each periodic angle before applying its sine–cosine encoding, avoiding a discontinuity at angle wrap-around. The displacement loss applies the same map to predicted and target configuration pairs relative to the predecessor $q(k)$. For a finite multiset $\mathcal{E}$ of scalar errors, define

$$\text{TopMean}_{0.25}(\mathcal{E}) = \frac{1}{|\lceil 0.25|\mathcal{E}| \rceil|} \sum_{z \in \text{Top}_{\lceil 0.25|\mathcal{E}| \rceil}(\mathcal{E})} z, \quad (17)$$

where $\text{Top}_r(\mathcal{E})$ returns the r largest elements with multiplicity. For each minibatch $\mathcal{B}$, $\mathcal{L}_{\text{tail}}(\mathcal{B}) = \text{TopMean}_{0.25}(\mathcal{E}(\mathcal{B}))$. The error multiset and component weights are specified separately for each source and task. Table 13 gives the complete contract. “Terminal configuration MSE” means one scalar per rollout record at $k = H$; the Reacher tail instead contributes one maximum joint-chord

Table 11: Module training contracts. Arrows denote consecutive stages; update counts and learning rates align from left to right. For OGBench-Cube, S1–S3 denote initial module training, joint Tokenizer–Grounder refinement, and height-balanced refinement.

Module	Initialization	Objective	Updates	LR ($\times 10^{-4}$)	Status
TwoRoom					
Tokenizer	random	RGB reconstruction + change support	100	2	frozen
Grounder	random head	port configuration + finite-difference regression	1,000	3	frozen
History module	random	causal token-dynamics NLL	500	3	frozen
Unified transition	random $\rightarrow$ carried	recursive $F \rightarrow (F + I)$ rollout	3,000 $\rightarrow$ 3,000	3 $\rightarrow$ 1	active
Reacher					
Pixel coder	en-random	reconstruction + equivariance + geometry coverage	500 $\rightarrow$ 2,000	2 $\rightarrow$ 0.5	frozen
Grounder	staged heads	physical, configuration, horizon, residual regression	1,500 $\rightarrow$ 2,000 $\rightarrow$ 3,000 $\rightarrow$ 3,000	2; 0.5/5; 2; 1/5	frozen
History module	random	causal representation dynamics	500	2	frozen
Unified transition	random $\rightarrow$ carried	recursive $I \rightarrow I \rightarrow (F + I)$ rollout	4,000 $\rightarrow$ 2,000 $\rightarrow$ 3,000 $\rightarrow$ 3,000	3 $\rightarrow$ 1 $\rightarrow$ 1	active
Push-T					
Tokenizer	pretrained	frontend position + visual preservation	500	1	frozen
Grounder	random head	typed configuration regression	2,000	3	frozen
History module	inherited parent	rollout objective	2,000	0.05	joint; active
Unified transition	recurrent parent	factual + common-reset + structured rollout	2,000	0.3	joint; active
OGBench-Cube					
Tokenizer	random	reconstruction + change support + token anchoring	S1: 500 S2: 1,000 S3: 2,000	S1: 2 S2: 0.1 S3: 0.05	frozen
Grounder	random head	pose regression + height-balanced tail	S1: 1,500 S2: 1,000 S3: 2,000	S1: 3 S2: 1 S3: 0.5	frozen
History module	random	causal token-dynamics NLL	1,000	3	frozen
Unified transition	random	recursive $(F + I)$ rollout	3,000	1	active

All stages use AdamW. Frontend and grounder checkpoints use held-out representation or configuration gates; recurrent checkpoints use held-out multi-horizon rollout gates.

Table 12: Complete recurrent-transition update schedules. Arrows separate consecutive stages; F and I denote factual and intervention sources. The final stage in every row uses Equation (9).

Environment	Active sources by stage	Updates by stage	$N_{\text{rec}}^{(e)}$
TwoRoom	$F \rightarrow (F + I)$	3,000 $\rightarrow$ 3,000	6,000
Reacher	$I \rightarrow I \rightarrow (F + I)$	4,000 $\rightarrow$ 2,000 $\rightarrow$ 3,000	9,000
Push-T	$(F + I) \times 6 \rightarrow \text{SR}$	5,000 $\rightarrow$ 3,000 $\rightarrow$ 3,000 $\rightarrow$ 5,000 $\rightarrow$ 10,000 $\rightarrow$ 3,000 $\rightarrow$ 2,000	31,000
OGBench-Cube	$F + I$	3,000	3,000

error per record and supervised macro index. The same decoder D_ρ is applied at $k = 0$ and at future steps; there is no separate initial-state decoder.

For a minibatch $\mathcal{B} = \{\xi_n\}_{n=1}^N$, predictions are generated by Equation (6); batch indices are suppressed below. Write $\text{MSE}(X, Y)$ for the mean squared error over every scalar entry of equally shaped tensors. Here $\hat{c}_0 = G_\psi(E_{\phi_E}(o_t, \mathbf{0}))$ is the configuration inferred from the current image, whereas c_0 is the ground-truth task configuration at the same time step, obtained from the recorded simulator state through the task-specific coordinate map in Appendix B. Define the anchored target by $\bar{c}_0 = \hat{c}_0$ and $\bar{c}_k = c_k$ for $k \geq 1$. The first displacement comparison therefore uses the same inferred anchor $\hat{c}_0$ on its predicted and target sides, preventing current-frame grounding error from being counted again as transition-increment error. The exact components grouped conceptually in Equation (8) are

$$\begin{aligned}
\mathcal{L}_c(\mathcal{B}) &= \text{MSE}(\hat{c}_{1:H}, c_{1:H}), \\
\mathcal{L}_{\text{disp}}(\mathcal{B}) &= \text{MSE}(\{\Lambda_\kappa(\hat{c}_k, \hat{c}_{q(k)})\}_{k=1}^H, \{\Lambda_\kappa(c_k, \bar{c}_{q(k)})\}_{k=1}^H), \\
\mathcal{L}_\chi(\mathcal{B}) &= \text{MSE}(\{D_\rho(\hat{m}_k)\}_{k=1}^H, \chi_{1:H}), \quad \mathcal{L}_{\chi,0}(\mathcal{B}) = \text{MSE}(D_\rho(\hat{m}_0), \chi_0). \quad (18)
\end{aligned}$$

For source $\sigma \in \{F, I\}$, the training loss is

$$\ell_\sigma(\mathcal{B}) = w_{\sigma,c}\mathcal{L}_c + w_{\sigma,\text{disp}}\mathcal{L}_{\text{disp}} + w_{\sigma,\text{tail}}\mathcal{L}_{\text{tail}} + w_{\sigma,\chi}\mathcal{L}_\chi + w_{\sigma,\chi_0}\mathcal{L}_{\chi_0}. \quad (19)$$

In the three-part presentation of Equation (8), the future-configuration group contains $\mathcal{L}_c$ and $\mathcal{L}_{\text{tail}}$, the configuration-displacement group contains $\mathcal{L}_{\text{disp}}$, and the auxiliary-dynamics group contains $\mathcal{L}_\chi$ and $\mathcal{L}_{\chi_0}$.

Table 13: Exact rollout-loss contract. Each row lists $(w_{\sigma,c}, w_{\sigma,\text{disp}}, w_{\sigma,\text{tail}}, w_{\sigma,\chi}, w_{\sigma,\chi_0})$ from Equation (19).

Task	Source σ	$(\kappa, q(k))$	Error multiset $\mathcal{E}(\mathcal{B})$ for $\mathcal{L}_{\text{tail}}$	component weights
TwoRoom	factual F	(emb, $k - 1$)	terminal configuration MSE	(2, 2, 0.5, 0.5, 0.5)
TwoRoom	intervention I	(emb, $k - 1$)	terminal configuration MSE	(2, 2, 1.0, 0.5, 0)
Reacher	factual F	(rel, 0)	all-horizon maximum joint chord error	(2, 2, 0.5, 0.5, 0.5)
Reacher	intervention I	(rel, 0)	all-horizon maximum joint chord error	(2, 2, 0.5, 0.5, 0.5)
Push-T	factual F	(emb, $k - 1$)	terminal configuration MSE	(2, 2, 0.5, 0.5, 0.5)
Push-T	intervention I	(emb, $k - 1$)	terminal configuration MSE	(2, 2, 1.0, 0.5, 0)
Push-T	structured refinement SR	(emb, $k - 1$)	terminal configuration MSE	(2, 2, 1.0, 0.5, 0.5)
OGBench-Cube	factual F	(emb, $k - 1$)	terminal configuration MSE	(2, 2, 0.5, 0.5, 0.5)
OGBench-Cube	intervention I	(emb, $k - 1$)	terminal configuration MSE	(2, 2, 1.0, 0.5, 0)

The Push-T structured-refinement minibatch mixes broad factual, IID common-reset, and contact-balanced structured records with source weights 1.0, 0.5, and 1.0. Contact metadata affects sampling only; reward, success, planner cost, and candidate rank are not training targets.

Cube public reproduction. Separate from the three-seed evaluation on 100 paired trials, the public seed-42 50-trial Cube list gives 42/50 success for FIRM and 35/50 for LeWM with 300/10/top-30 CEM. This reproduction is not averaged into Table 2.

F FULL-CHAIN TRAINING-SOURCE ABLATIONS

The first three rows of Table 4 retrain the complete Reacher and Cube task pipelines from independent initialization under factual-only, intervention-only, or combined supervision. Within each environment, each variant uses three random seeds while holding the architecture, stage-wise optimization schedule, data split, and paired evaluation trials fixed. Figure 3 shows the distributions across independently initialized training runs. The two additional single-seed diagnostics are Reacher-only; the mismatched-outcome diagnostic preserves the marginal sets of histories, actions, and outcomes while destroying their correspondence.

G FULL PER-SEED RESULTS

Table 14: Per-seed paired results. Δ is FIRM minus LeWM in success points; intervals are 95% paired bootstrap intervals. Rescue/harm count trials won only by FIRM / only by LeWM. LeWM timing is fixed within an environment because its checkpoint and paired evaluator are reused.

Environment	Seed	FIRM	LeWM	Δ [95% CI]	Rescue / harm	FIRM sec.	LeWM sec.
TwoRoom	1	99	89	+10 [5,16]	10 / 0	0.480	3.224
	2	100	89	+11 [5,18]	11 / 0	0.378	3.224
	3	98	89	+9 [3,16]	10 / 1	0.391	3.224
Reacher	1	95	88	+7 [0,15]	11 / 4	0.675	5.550
	2	92	88	+4 [-4,12]	10 / 6	0.407	5.550
	3	91	88	+3 [-5,11]	10 / 7	0.429	5.550
Cube	1	88	70	+18 [8,28]	23 / 5	0.771	2.389
	2	85	70	+15 [5,25]	21 / 6	0.575	2.389
	3	91	70	+21 [13,30]	22 / 1	0.560	2.389
Push-T	1	72	90	-18 [-27,-9]	3 / 21	1.656	3.926
	2	74	90	-16 [-25,-7]	4 / 20	1.795	3.926
	3	81	90	-9 [-18,0]	7 / 16	2.128	3.926

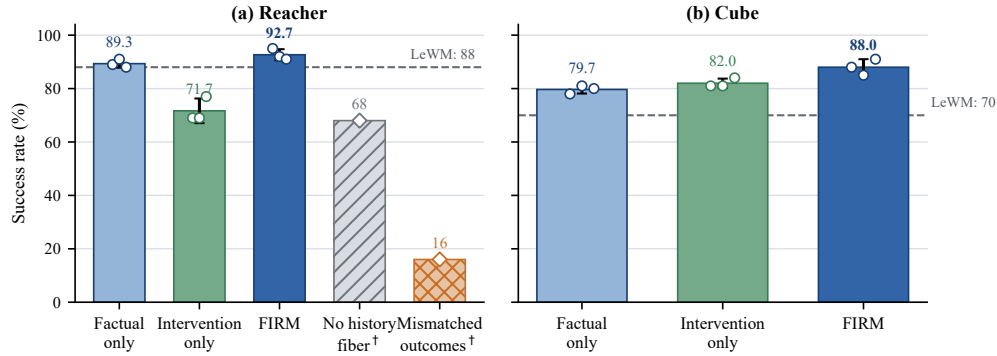


Figure 3: **Full-chain training-source ablations.** Each panel shows three independently initialized training runs (dots), their mean, and sample s.d. for factual-only, intervention-only, and combined training. The combined pipeline performs best in every run on both tasks. Reacher additionally includes two separate single-seed diagnostics marked $\dagger$. All results use released-original CEM; dashed lines show the task-specific fixed LeWM paired baselines.

All TwoRoom and Cube intervals are positive. Reacher favors FIRM in every seed, with two intervals crossing zero. Push-T consistently favors LeWM across all three random seeds, separating a persistent-contact limitation from random-seed variation.

All values in Tables 3 and 14 come from the paired evaluator on one local evaluation host equipped with an NVIDIA GeForce RTX 3060 Laptop GPU (6 GB). A timer brackets repeated online planner work: current/goal encoding required by the evaluator, recurrent candidate rollout, CEM population updates, and action selection. It excludes offline cache construction, dataset loading, training, process launch, and simulator creation outside the repeated boundary. Both methods use the same paired evaluation trials and CEM budget in each task.

H DATA AND EVALUATION PROTOCOL

Table 15 summarizes the data used in each environment. We train three complete models per environment from independent random initializations while holding the data split, intervention set, and hyperparameters fixed across seeds.

Table 15: Dataset statistics for the final joint recurrent stage. A factual rollout segment contains a three-frame history and its recorded 25-control continuation. Intervention states are the recorded interface states used to initialize action branches; total branch rollouts equal intervention states times action branches per state. All three models in an environment are evaluated on the same 100 paired trials.

Environment	Training episodes	Validation episodes	Factual rollout segments	Intervention states	Action branches per state	Branch rollouts	Evaluation trials
TwoRoom	9,000	1,000	16,384	128	32	4,096	100 paired trials
Reacher	9,000	1,000	16,384	512	16	8,192	100 paired trials
OGBench-Cube	9,000	1,000	4,096	128	16	2,048	100 paired trials
Push-T	16,685	2,000	16,384	512	16	8,192	100 paired trials

I LLM USAGE STATEMENT

A large language model was used as a writing assistant to help organize the manuscript, improve prose, and check consistency between claims and project artifacts. It did not generate experimental measurements. The authors are responsible for verifying every equation, citation, result, and scientific claim in the submitted version.